

NumPy

arrays, indexing, vectorization, aggregation

DAEN 300

Alexander Abuabara

Fall 2026



References

https:

[//numpy.org/doc/stable/user/absolute_beginners.html](https://numpy.org/doc/stable/user/absolute_beginners.html)

<https://madewithml.com/courses/foundations/numpy/>

Learning Objectives

1. Explain why NumPy arrays are useful for data analysis.
2. Create and inspect one- and multi-dimensional arrays.
3. Access data using indexing and slicing.
4. Perform vectorized computations on arrays.
5. Compute summary statistics along different axes.
6. Explain the basic idea of broadcasting.
7. Use Boolean masks to select data.

Why Do We Need Numerical Arrays?

Many datasets can be represented as arrays of numbers.

Examples

- ▶ Sensor measurements
- ▶ Images
- ▶ Audio signals
- ▶ Financial observations
- ▶ Experimental measurements
- ▶ Machine-learning features

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

Rows often represent observations.

Columns often represent variables.

Python Lists vs. NumPy Arrays

A Python list is extremely flexible:

```
values = [10, "hello", 3.14, True]
```

A NumPy array normally stores elements of a common data type:

```
import numpy as np  
  
x = np.array([10, 20, 30, 40])
```

NumPy sacrifices some of the flexibility of Python lists in exchange for efficient storage and fast numerical computation.

The NumPy ndarray

The central NumPy object is the multidimensional array:

`numpy.ndarray`

```
import numpy as np

x = np.array([5, 10, 15, 20])
```

Useful attributes include:

Attribute	Meaning
<code>x.ndim</code>	Number of dimensions
<code>x.shape</code>	Size of each dimension
<code>x.size</code>	Total number of elements
<code>x.dtype</code>	Data type of elements
<code>x.nbytes</code>	Memory used by array data

Creating NumPy Arrays

```
import numpy as np

a = np.array([1, 2, 3, 4])

zeros = np.zeros(5)

ones = np.ones((2, 3))

sequence = np.arange(0, 10, 2)

grid = np.linspace(0, 1, 6)

identity = np.eye(3)
```

Creating NumPy Arrays

```
import numpy as np

a = np.array([1, 2, 3, 4])

zeros = np.zeros(5)

ones = np.ones((2, 3))

sequence = np.arange(0, 10, 2)

grid = np.linspace(0, 1, 6)

identity = np.eye(3)
```

Question

What are the shapes of `ones` and `identity`?

Dimensions and Shapes

Consider:

```
A = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print(A.ndim)
print(A.shape)
print(A.size)
```

The result is conceptually:

$$A = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$$

Therefore,

$$\text{ndim} = 2, \quad \text{shape} = (2, 3), \quad \text{size} = 6.$$

Indexing Arrays

For a one-dimensional array:

```
x = np.array([10, 20, 30, 40])  
x[0] # 10  
x[2] # 30  
x[-1] # 40
```

For a two-dimensional array:

```
A = np.array([  
    [10, 20, 30],  
    [40, 50, 60]  
)  
A[0, 1] # 20  
A[1, 2] # 60
```

Python and NumPy use zero-based indexing.

Slicing Arrays

The general slicing pattern is `x[start:stop:step]`.

Example:

```
x = np.arange(10)
x[:4] # first four values
x[4:] # values beginning at index 4
x[2:7] # indices 2 through 6
x[::2] # every second value
x[::-1] # reverse
```

For matrices:

```
A[:2, :3]
```

means: first two rows, first three columns.

Reshaping Arrays

Arrays often need to be reorganized.

```
x = np.arange(12)
A = x.reshape(3, 4)
print(A)
```

Conceptually,

$$[0, 1, 2, \dots, 11] \longrightarrow \begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{bmatrix}$$

Requirement

The total number of elements cannot change.

$$12 = 3 \times 4$$

The Big Idea: Vectorization

Suppose we want to square every value.

Loop-oriented thinking:

$$y_i = x_i^2$$

for every individual i .

Array-oriented thinking:

$$\mathbf{y} = \mathbf{x}^2$$

Vectorization

Describe the computation using operations on entire arrays rather than manually iterating through individual elements.

Loop vs. Vectorized Computation

Python loop:

```
x = np.array([1, 2, 3, 4])  
  
result = np.empty(len(x))  
  
for i in range(len(x)):  
    result[i] = x[i] ** 2
```

NumPy:

```
result = x ** 2
```

Both implement $[1, 2, 3, 4] \rightarrow [1, 4, 9, 16]$.

Vectorized NumPy operations move repetitive numerical work into optimized compiled routines.

Universal Functions

NumPy calls many vectorized element-wise operations **universal functions**, or **ufuncs**.

```
x = np.array([1, 2, 3, 4])
```

```
x + 10
```

```
x * 2
```

```
x ** 2
```

```
np.sqrt(x)
```

```
np.exp(x)
```

```
np.log(x)
```

```
np.sin(x)
```

For example,

$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} + 10 = \begin{bmatrix} 11 & 12 & 13 & 14 \end{bmatrix}.$$

Aggregations

Often we want to summarize an entire dataset.

```
x = np.array([5, 10, 15, 20])  
  
x.sum()  
x.mean()  
x.min()  
x.max()  
x.std()
```

Common functions include:

sum, mean, median, min, max, variance, standard deviation.

Understanding axis

Consider:

$$A = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$$

```
A.sum()
```

adds every value.

Understanding axis

Consider:

$$A = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$$

```
A.sum()
```

adds every value.

```
A.sum(axis=0)
```

produces column sums:

[50, 70, 90].

Understanding axis

Consider:

$$A = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$$

```
A.sum()
```

adds every value.

```
A.sum(axis=0)
```

produces column sums:

[50, 70, 90].

```
A.sum(axis=1)
```

produces row sums:

[60, 150].

Broadcasting

Broadcasting allows NumPy to perform operations involving arrays with different, but compatible, shapes.

For example,

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} + 10 = \begin{bmatrix} 11 & 12 & 13 \end{bmatrix}.$$

Conceptually, NumPy treats the scalar as though it were

$$\begin{bmatrix} 10 & 10 & 10 \end{bmatrix}.$$

NumPy does not necessarily create all of these duplicated values in memory. Broadcasting describes how the operation behaves.

Broadcasting a Vector Across a Matrix

Let

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad b = [0 \quad 10 \quad 20].$$

Then:

```
A = np.ones((3, 3))  
b = np.array([0, 10, 20])
```

A + b

produces

$$\begin{bmatrix} 1 & 11 & 21 \\ 1 & 11 & 21 \\ 1 & 11 & 21 \end{bmatrix}.$$

Boolean Comparisons

Comparisons can be applied to an entire array.

```
scores = np.array([62, 91, 75, 48, 88])  
  
scores >= 70
```

Result:

[False, True, True, False, True]

This Boolean array is called a **mask**.

Boolean Masking

A mask can select values satisfying a condition.

```
scores = np.array([62, 91, 75, 48, 88])  
  
passing = scores[scores >= 70]  
  
print(passing)
```

Result:

[91, 75, 88]

We can also count observations satisfying a condition:

```
np.sum(scores >= 70)
```

Result:

3

Example: Student Scores

Suppose each row is a student and each column is an exam.

```
scores = np.array([
    [80, 72, 91],
    [65, 70, 68],
    [92, 88, 95],
    [75, 81, 78]
])
```

Question 1: What is each student's mean score?

```
student_means = scores.mean(axis=1)
```

Question 2: What is the mean for each exam?

```
exam_means = scores.mean(axis=0)
```

Example: Centering the Data

Suppose we want to measure each score relative to its exam mean.

```
exam_means = scores.mean(axis=0)

centered = scores - exam_means
```

Mathematically,

$$x_{ij}^* = x_{ij} - \bar{x}_j.$$

What made this simple?

1. Aggregation with `mean(axis=0)`
2. Vectorized subtraction
3. Broadcasting

Common NumPy Mistakes

1. **Confusing shape and size**

$$(100, 5) \neq 500$$

2. **Using the wrong axis**
3. **Forgetting that array elements have a fixed dtype**
4. **Writing unnecessary Python loops**
5. **Assuming every pair of shapes can broadcast**
6. **Forgetting zero-based indexing**

In-Class Exercise

Create the following matrix:

$$X = \begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \\ 14 & 16 & 18 \end{bmatrix}.$$

Using NumPy:

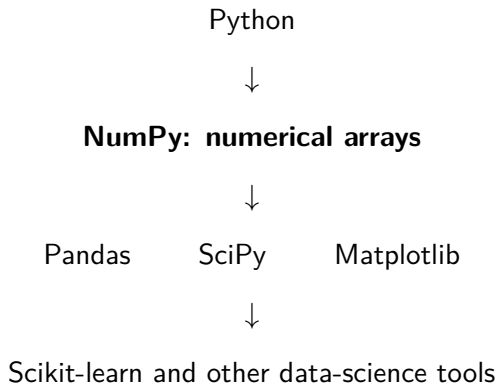
1. Find its shape.
2. Extract the second row.
3. Extract the third column.
4. Compute the mean of each column.
5. Select all values greater than 10.
6. Multiply every value by 2 without using a loop.
7. Subtract each column's mean from that column.

Solution

```
X = np.array([
    [2, 4, 6],
    [8, 10, 12],
    [14, 16, 18]
])
X.shape
X[1, :]
X[:, 2]
X.mean(axis=0)
X[X > 10]
2 * X
X - X.mean(axis=0)
```

NumPy in Data Science

Understanding arrays makes many higher-level Python data tools easier to understand.



Key Points

1. NumPy's basic data structure is the `ndarray`.
2. Arrays have dimensions, shapes, and fixed data types.
3. Indexing and slicing provide efficient access to subsets.
4. Vectorization replaces many explicit Python loops.
5. Aggregations summarize data efficiently.
6. The `axis` argument controls which dimension is reduced.
7. Broadcasting enables operations across compatible shapes.
8. Boolean masks provide concise conditional data selection.

Discussion

1. Why might a NumPy array use less memory than a Python list?
2. Why can vectorized operations outperform Python loops?
3. What does `axis=0` mean for a two-dimensional array?
4. When might broadcasting be useful?
5. How could Boolean masking replace an explicit loop?